SIC Lab
Digital signatures
version 1.0

Authors: André Zúquete, Vitor Cunha

Version log:

- 1.0: Initial version

1 Introduction

In this guide we will develop programs that use cryptographic methods, relying in the **crypto** library.

This lab makes use of the commands for creating key pairs developed in the previous lab. # RSA

1.1 RSA key pair generation

Create 4 RSA key pairs with the following key length: 1024, 2048, 3072 or 4096.

1.2 RSA signature creation

Create a program to sign a file using the RSA algorithm. You can use as a starting point the `rsa_sign.c` file. The user must indicate the following parameters:

1. Name of the file with the private key to produce the signature;
2. A padding scheme (either PKCS1 or PSS);
3. A digest function (SHA-256, SHA-348, SHA-512, etc.);
4. Name of the file to sign (or stdin if missing);
5. Name of the signature file (or stdout if missing).

For the same file, use different combinations of private keys, padding algorithms and hashing functions.

PKCS1, or more precisely PKCS1_v1.5, is the oldest padding scheme, which is used very often, but has been progressively abandoned in favor of PSS (Probabilistic Signature Scheme). PSS is a randomized computation, which means that its computation uses a random value. Consequently, it never produces the same output for the same inputs (excluding that random value).

You can save the signature data both in a binary format (the one produced by RSA, and the one being used in the example provided) or in a textual encoding. Base64 is a good choice.

```
#include <openssl/evp.h>

uint8_t * signature;
int signature_len;
uint8_t * encoded_signature;
int encoded_signature_len;

signature = ...
signature_len = ...

/*
 * Base64 output length
```

```
 * Each 3 bytes are converted to 4
 * Padding is used for trailing 1 or 2 bytes
 * The output is meant to be a string, therefore we need room for a NULL character
*/
encoded_signature_len = 4 * ((signature_len + 2) / 3);
encoded_signature = malloc( 1 + encoded_signature_len );

EVP_EncodeBlock( encoded_signature, signature, signature_len );
```

For validating a signature encoding in Base64, you need to do the opposite oporation prior to the verification:

```
#include <openssl/evp.h>

uint8_t * signature;
uint8_t * encoded_signature;
int signature_len;
int encoded_signature_len;

encoded_signature = ...
encoded_signature_len = strlen( encoded_signature );

signature_len = 3 * encoded_signature_len / 4; // We may be allocating 1 or 2 more bytes than needed... but we need to!
signature = malloc( signature_len );

signature_len = EVP_DecodeBlock( signature, encoded_signature, encoded_signature_len );

// Correct the useful length of the signature by observing the padding symbols used, == (-2) or = (-1)
if (encoded_signature[encoded_signature_len - 1] == '=') signature_len--;
if (encoded_signature[encoded_signature_len - 2] == '=') signature_len--;
```

Questions:

- What are the parameters that have impact on the dimension of the signature?
- Sign the same file two times with the same combination of parameters and save the two signatures in two files. Check if they are equal (use the 'cmp' command). Change the padding scheme and check again. In which case does the signature change, and why?

1.3 RSA signature verification

Create a program to verify a signature of a file using the RSA algorithm. The user must indicate the following parameters:

1. Name of the file with the public key to verify the signature;
2. The padding scheme that was used;
3. The digest function that was used (SHA-256, SHA-348, SHA-512, etc.);
4. Name of the signed file;
5. Name of the signature file (or stdin if missing).

Questions:

- What happens, when you do not use the correct parameters to validate a signature?
- What happens, when you update the original final and validate the signature afterwords?

2 Elliptic curves

2.1 ECC key pair generation

In the last lab we created an application to produce a password-protected P-251 key pair. Add the curve selection to the program, in order to be able to use different curves.

Create 3 EC key pairs with the following curves: P-521, B-409, K-163.

2.2 EC signature creation

Create a program to sign a file using an EC key pair. You can use as a starting point the `ec_sign.c` file (which is not prepared to handle the password that protects a private key!). The user must indicate the following parameters:

1. Name of the file with the private key to produce the signature;
2. The password used to protect the private key;
3. A digest function (SHA-256, SHA-348, SHA-512, etc.);
4. Name of the file to sign (or stdin if missing);
5. Name of the signature file (or stdout if missing).

For the same file, use different combinations of private keys and hashing functions.

EC signatures do not use a padding scheme (because they are not based on cipher/decipher operations). They use an algorithm known as ECDSA (Elliptic Curve Digital Signature Algorithm). This produces two values that form the signature.

ECDSA is a randomized computation, which means that its computation uses a random value. Consequently, it never produces the same output for the same inputs (excluding that random value).

Questions:

- What are the parameters that have impact on the dimension of the signature?
- Sign the same file two times with the same combination of parameters and save the two signatures in two files. Check if they are equal (use the 'cmp' command). Change the padding scheme and check again. In which case does the signature change, and why?

2.3 EC signature verification

Create a program to verify a signature of a file using the EC algorithm. The user must indicate the following parameters:

1. Name of the file with the public key to verify the signature;
2. The digest function that was used (SHA-256, SHA-348, SHA-512, etc.);
3. Name of the signed file;
4. Name of the signature file (or stdin if missing).

Questions:

- What happens, when you do not use the correct parameters to validate a signature?
- What happens, when you update the original final and validate the signature afterwords?

3 Further Reading

2. SafeCurves